NetScope IPv6 Auditor

Problem Statement PS-027: IPv6 Website and Dependency Readiness Auditor

Category: Internet Measurement

Project Metadata

- **Project Name:** NetScope IPv6 Auditor
- **Tagline:** Beyond DNS. Measure Every Dependency. Prove IPv6 Readiness.
- **Problem Statement ID:** PS-027
- **Category:** Internet Measurement
- **Inspiration & Reference:** IETF webres6 initiative (https://github.com/SAP/webres6)
- **Primary Deliverables:** Full-Stack Application, Dependency Graph, Controlled Lab, 30-Site Benchmark, CSV/JSON Reports, Pseudocode Snippet, and Demo Script.

1. Executive Summary & Problem Definition

1.1 The Dual-Stack Illusion

Global IPv6 adoption now exceeds 45%, and major cellular networks (such as Reliance Jio and T-Mobile) operate purely on IPv6 cores with 464XLAT translation. However, enterprise websites and public portals often claim "IPv6 compliance" based solely on a single DNS metric: the presence of an AAAA record on their apex domain.

This creates a dangerous blindspot. Modern web pages are composite applications loading dozens of third-party assets:

- External JavaScript bundles (analytics, tag managers, CDN libraries)
- Cascading Style Sheets (CSS)
- Web fonts and iconography
- Media files and images
- Dynamic API endpoints (`fetch` / `XHR`) and embedded frames

1.2 The Happy Eyeballs Masking Effect (RFC 8305)

Standard desktop browsers implement **RFC 8305 (Happy Eyeballs v2)**, which races IPv4 and IPv6 connection attempts concurrently. If an IPv6 path drops SYN packets or encounters a 250ms delay, the browser silently switches to IPv4 without raising any visible alert. Consequently:

- Developers believe their site is IPv6-ready because it works on desktop.
- When an IPv6-only client visits the site, un-translated dependencies break catastrophically.

1.3 Project Goal

Inspired by the **IETF webres6** initiative, NetScope IPv6 Auditor provides an automated, browser-integrated network auditing platform that discovers runtime dependencies, enforces discrete family-isolated socket probes (DNS, TCP, TLS, and HTTP), maps an interactive dependency graph, and verifies compliance against a controlled test laboratory.

---

2. System Architecture & Methodology



---

3. The 7 Core Scientific Axioms

1. **Browser-Observed Discovery:** NetScope runs Playwright Chromium to intercept DOM and runtime network requests rather than static HTML regex parsing.

2. **The AAAA Fallacy:** An AAAA DNS record proves only domain configuration; it does not guarantee that TCP port 443 is open, bound on `[::]`, or admitted by cloud firewalls.

3. **Address Family Isolation:** Circumvents Happy Eyeballs by opening discrete raw OS sockets explicitly configured with `family: 4` and `family: 6`.

4. **NAT64 / DNS64 Detection:** Automatically filters synthetic `64:ff9b::/96` addresses (RFC 6052) so ISP translation gateways are not mistaken for native authoritative AAAA records.

5. **HTTP 404 Disassociation:** Confirms that application-layer 404 Not Found responses represent successful network transport (DNS, TCP, and TLS all passed).

6. **CORS Disassociation:** Distinguishes browser Same-Origin Policy header restrictions from IP packet drops.

7. **Denominator Integrity:** Excludes unmeasured or unsupported resources from success rate denominators to prevent metric skew.

---

4. Controlled Test Laboratory (Ground-Truth Suite)

The Controlled Lab server runs on port 5005, serving 8 deterministic test cases specified in `manifest.json`:

| Test ID | Path | Type | Expected IPv4 | Expected IPv6 | Expected Overall | Scientific Principle Tested |

|---|---|---|---|---|---|---|

| **TC-01** | `/api/dual-stack-pass` | fetch | Passed | Passed | **Passed** | Positive control for healthy dual-stack service |

| **TC-02** | `/api/http-404-error` | fetch | Passed | Passed | **Passed** | Section 6 Rule: HTTP 404 is NOT a transport failure |

| **TC-03** | `/api/http-500-error` | fetch | HTTP error | HTTP error | **HTTP error** | Application 5xx distinguished from network drops |

| **TC-04** | `/api/cors-restricted` | fetch | Passed | Passed | **Passed** | CORS restriction is not an IPv6 transport failure |

| **TC-05** | `/api/ipv4-only-sim` | fetch | Passed | DNS failure | **DNS failure** | Seeded IPv4-only resource (Missing AAAA) |

| **TC-06** | `/api/broken-ipv6-tcp` | fetch | Passed | TCP failure | **TCP failure** | AAAA published, but IPv6 TCP SYN dropped / reset |

| **TC-07** | `/style.css` | stylesheet | Passed | Passed | **Passed** | Dual-stack CSS asset delivery verification |

| **TC-08** | `/app.js` | script | Passed | Passed | **Passed** | Dual-stack JavaScript bundle verification |

Confusion Matrix Validation Results:

- **Accuracy:** 100%
- **Precision:** 100%
- **Recall:** 100%
- **F1-Score:** 100%
- **False Positives:** 0 | **False Negatives:** 0

---

5. Thirty-Site Industry Benchmark Findings

NetScope includes a multi-sector benchmark cohort across 5 key verticals:

1. **Search & Cloud:** Google, Cloudflare, Microsoft, AWS, Wikipedia, Fastly.

2. **Tech & Developer:** GitHub, Stack Overflow, Python PSF, Mozilla, NPM Registry, Docker.

3. **Media & News:** BBC News, CNN, The New York Times, The Guardian, Reuters, Reddit.

4. **E-commerce & Retail:** Amazon, eBay, Apple, Walmart, Shopify, Etsy.

5. **Government & Education:** NIST, NASA, MIT, Stanford University, CERN, W3C.

Key Finding: Even among tier-1 tech websites, apex domains like `github.com` lack AAAA records, while their CDN dependencies (`githubassets.com`) are fully dual-stack. NetScope visually demonstrates this disparity in the React Flow graph with distinct green (Passed) and red (Failed) nodes.

---

6. Pseudocode Snippet for Mentor Demonstration

```
def audit_website_dependencies(target_url):
    # Step 1: Pre-flight security & SSRF defense
    validate_url_against_private_subnets(target_url, blocked=[RFC1918, RFC4291, RFC3927])

    # Step 2: Headless browser discovery (Playwright)
    browser = playwright.launch_chromium(headless=True)
    page = browser.navigate(target_url, wait_for="load")
    dependencies = page.capture_all_network_and_dom_resources()
    browser.close()

    # Step 3: Independent address-family socket measurements
    for resource in dependencies:
        dns = query_dns(resource.host) # Extracts A and AAAA records (excludes 64:ff9b::)

        # Test IPv4 (Enforced family: 4)
        v4_result = open_tcp_and_tls_socket(ip=dns.a_records[0], family=4, port=resource.port)

        # Test IPv6 (Enforced family: 6)
        if not dns.has_aaaa():
            v6_result = Result(outcome="DNS failure", reason="Missing AAAA record")
        else:
            v6_result = open_tcp_and_tls_socket(ip=dns.aaaa_records[0], family=6, port=resourc
e.port)

        # Step 4: Classify failure stage and generate DevOps recommendation
        classification = classify_parity(dns, v4_result, v6_result)
        save_measurement_record(resource, classification, geo_context=lookup_asn(dns))

    return generate_dependency_graph_and_reports()
```

---

7. Deliverables Checklist & Compliance

- [x] **Browser-Based Auditor Web Application:** Live on `http://localhost:3000` with Express backend on `http://127.0.0.1:5000`.
- [x] **Interactive Dependency Graph:** React Flow visualization with zoom/pan, filters, and inspection drawer.
- [x] **30-Site Selected Benchmark:** Configurable batch runner across 30 public domains.
- [x] **Controlled Test Site:** Port 5005 test server with seeded IPv4-only and broken TCP assets.
- [x] **Failure Stage Isolation:** Categorizes failures into DNS, TCP, TLS, and HTTP.
- [x] **Geolocation & Resolver Metadata:** Recorded in database, JSON, and CSV exports.
- [x] **CSV/JSON Measurement Reports:** Export endpoints matching RFC 4180.
- [x] **PDF Submission Document:** Ready in `Proposed-structure-hackathon.md` (convertible to PDF).
- [x] **27 Passing Automated Tests:** 100% test pass rate across backend, frontend, and E2E suites.
- [x] **Reference to IETF webres6:** Credited in architecture, reports, and methodology.